

Darmstadt University of Applied Sciences

– Faculty of Computer Science –

AI driven evolution of penetration testing in software security

by

Henning Prien

Matriculation number: 1132644

First Examiner : Dominik Machold

DECLARATION

I hereby certify that I have written the present paper independently and have not used any sources other than those listed in the bibliography.

All passages that have been taken literally or in substance from published or unpublished sources are identified as such.

The drawings or illustrations in this work were created by myself or are provided with an appropriate source reference.

This work has not been submitted to any other examination authority in the same or a similar form.

Darmstadt, March 16, 2026

Henning Prien

ABSTRACT

Software forms the backbone of modern society, supporting critical infrastructure such as communication networks, financial systems, and healthcare services. The security of these systems is a pressing concern, as vulnerabilities can lead to severe consequences, including financial losses, reputational harm, and even threats to human safety. While software vendors are motivated by economic incentives and legal obligations—such as compliance with regulations like the EU's General Data Protection Regulation (GDPR)—to prioritize security, the rapidly evolving threat landscape presents significant challenges. Emerging threats like advanced persistent attacks, ransomware, and supply chain vulnerabilities require more dynamic and scalable security solutions than traditional methods can offer.

This thesis investigates the potential of Artificial Intelligence (AI)-driven penetration testing, specifically through the use of Large Language Model (LLM)s, to automate and improve the process of identifying software vulnerabilities. The study begins by situating penetration testing within the Software Development Livecycle (SDLC), highlighting its role as a creative and adaptive method for simulating real-world cyberattacks. It then explores the transition from manual testing to AI-assisted approaches, analyzing recent developments such as PentestGPT, PentestAgent, and ARTEMIS. These advancements demonstrate that LLMs, when combined with domain-specific knowledge, can effectively perform penetration tests in controlled settings.

However, real-world applications reveal certain limitations. Tools like XBOW, despite their success in bug bounty programs, often generate high rates of false positives and struggle with proprietary or niche technology stacks. Security experts note that while AI-driven tools excel at identifying known vulnerabilities, they lack the lateral thinking and contextual adaptability that human testers provide. Additionally, security risks such as data leakage through prompt injection and biases in training data pose further challenges.

The thesis concludes that AI can substantially accelerate penetration testing, but it is not yet a substitution for human expertise. Future research should focus on enhancing AI's ability to navigate proprietary systems, reducing false positives, and addressing security risks. By overcoming these challenges, AI-driven penetration testing can become a dependable and scalable solution, complementing and ultimately strengthening security efforts.

CONTENTS

I Thesis

1 Introduction 2
- 1.1 Problem Statement . . . 2
- 1.2 Goal of research . . . 2
- 1.3 Research Question and Hypotheses . . . 3
- 1.4 Structure . . . 3
- 1.5 Methodology . . . 3

2 State of Research and related Work 4

3 Measures of Software Security 5

4 Penetration Testing 7

5 AI driven penetration testing 8
- 5.1 Security Challenges . . . 10

6 Discussion 11

7 Conclusion and Outlook 12
- 7.1 LLMs are capable of performing automated penetration tests . 12
- 7.2 Hypotheses evaluation . . . 12
- 7.3 Functional limitations . . . 12
- 7.4 Future Work . . . 12

II Appendix

Bibliography 15

ABBREVIATIONS

SDLC Software Development Livecycle

SAST Static Application Security Test

DAST Dynamic Application Security Test

CIA Confidentiality Indtegrity Availability

AI Artificial Intelligence

LLM Large Language Model

CTF Capture The Flag

CLI Command Line Interface

RAG Retrieval-augmented generation

GDPR General Data Protection Regulation

Part I

THESIS

1 INTRODUCTION

Software is integral to wide-ranging aspects of daily life, underpinning communication technologies, payment services, and health systems. Failures, unavailability, or data leaks in these crucial services can lead to severe harm or even death. Consequently, society has a significant indirect interest in software security due to its heavy reliance on digital infrastructure. [1]

Beyond societal concerns, software vendors also have compelling interests in providing secure software. On the one hand, companies can financially benefit from voluntarily disclosing their security measures. This transparency and proactive approach can positively influence their market value. [2]

On the other hand, software vendors are mandated to comply with various data protection laws and regulations. Non-compliance can result in severe penalties, including fines of up to 20,000,000€ limited to 4% of their total worldwide annual turnover. [3]

1.1 PROBLEM STATEMENT

These needs of cyber security need to be satisfied while surrounded by a dynamically and fast evolving threat landscape. State and non-state actors drive a wide and diverse landscape of cyber security threats. Supply chain attacks, advanced persistent threats and vulnerabilities on the Internet of Things are just some cyber security threats modern systems face. [4] Thus developing and maintaining software systems securely and protecting them from any harm through cyber criminals gets harder ever since.

1.2 GOAL OF RESEARCH

To address these growing difficulties of securing software systems, corporate vendors and developers can apply several security measures.

The SDLC describes multiple steps in which the development of secure software can be split up. One central part of the development of secure software is testing. [5] A highly effective approach of software testing is penetration testing [6]. Penetration testing used to be a manual and creative approach of dynamically testing for security vulnerabilities in software. However, traditional thus manual methods of penetration testing are no longer sufficient to keep track of fast evolving threat landscapes and ever growing software systems. This raises the need of automating and speeding up penetration testing. [7]

This term paper will discuss how AI-driven systems can support penetration testers. It will also discuss to which degree penetration testing already

can be automate and thus help penetration testers to focus of more advances research and testing.

1.3 RESEARCH QUESTION AND HYPOTHESES

The central research question addressed in this term paper is following: RQ: Are LLMs capable of performing automated penetration tests in corporate environments?

The hypotheses to answer this question is following: H: LLMs are capable of performing automated penetration tests on corporate and proprietary applications.

1.4 STRUCTURE

First of all, a foundation will be built up to give the content of this term paper a context. This will be done by the definition of important and related terms. Following, related historic approaches as well as related work will be presented in the context of the research question.

The main content of this term paper will consist of tree parts. To emphasize the role of penetration testing in secure software development, the first step will be to differentiate between measures of software security. Second step to answer the research question will be the introduction to penetration testing. Followed by third, taking a look at AI-driven penetration testing.

In the conclusion all information will be summarized and related to each other. In this part also the research question will be answered based on the findings.

1.5 METHODOLOGY

A literature review will discuss past and ongoing research on the automation of penetration testing. Capabilities of earlier solutions will be compared to the current state of research.

The usage of LLMs for automated penetration testing currently evolves fast. This is why special attention will be drawn to security experts' reports of experience. As these experts actually working with these tools and thus know their capabilities. This will ensure to cover the most recent evolutions in the field of automated penetration testing.

Based on the literature review, attention will also be drawn to security challenges and the need of future work.

2 STATE OF RESEARCH AND RELATED WORK

This term paper is partially based on research papers that are still in preprint or have not received much attention yet because they have been published recently. As the technology of AI and especially LLMs is still evolving and heavily underlies broad attention, some aspects might change and evolve in future.

There are three main differences regarding the knowledge based on which the penetration test is performed. Black-, Gray- and White-Box penetration testing. Black-Box penetration testing means that the penetration tester has no internal knowledge about the System Under Test. White-Box Testing means the Tester has complete knowledge of the internals and Grey-Box refers to partial knowledge of internal structures. [8]

Traditional penetration testing tooling already implemented various automation to support penetration testers [9].

Earlier approaches to automate the process of penetration testing leveraged Reenforcement Learning [10]. However scaling these approaches to an production ready state brought up difficulties [11].

Leveraging LLMs for penetration testing shows better results and scalability. The project PentestGPT also describes an agentic approach for orchestrating LLMs beneficially. [12]

Multiple tools already implement AI for penetration testing to support human penetration testers [13].

3 MEASURES OF SOFTWARE SECURITY

Multiple models and frameworks suggest steps that can be taken to develop secure software. They all describe different planning and implementation phases to cover every step of software engineering. [14]

The SDLC is one of these processes that allows the design, testing and implementation of secure software. It has received quite broad attention and many vendors use it to guideline their software development. The SDLC divides the software development into 6 steps:

1. Requirements and use cases
2. Design
3. Test plans
4. Code
5. Test results
6. Field Feedback

As this framework describes a live cycle, this process needs to be repeated when finished to implement the gathered feedback. The SDLC already highlights the importance of software testing although security measures can be taken in each step of the SDLC. [5]

A Systematic Mapping Study on Security Approaches in Secure Software Engineering was conducted in 2021. It revealed that many earlier conducted studies suggest to implement security measures in the design phase of the SDLC. The second most popular suggestion is to implement these measures through out the whole SDLC. Third, 23% suggest to implement security measures within the testing phase of the SDLC. [14] This clearly demonstrates the importance and popularity of implementing software security measures throughout the SDLC.

In the following, different software testing methods will be presented as the goal of this term paper is to evaluate how AI supports penetration testing. Afterwards the focus will be put to penetration testing as one of these testing techniques.

Already before a softwares ran once a Static Application Security Test (SAST) scanner is able to locate vulnerabilities in it. It can detect vulnerabilities in the source code itself as well as in byte or machine code. The weakness of these scanners is that they tend to have high false positive rates. [15]

Real attack scenarios like the exploitation of XSS and SQL injection are performed by Dynamic Application Security Test (DAST) scanners. DAST scanners have no access to the source code of an application neither to any run

time information like logs. [16] They systematically search for standard vulnerabilities like insecure headers and unsecured endpoints. Because of this rather unadapted test strategy they are not capable of finding logic errors or performing creative attacks.

4 PENETRATION TESTING

"[P][...]enetration testing is a detailed procedure to unravel [...] open doors in a system or network." [6] The technique of penetration testing is a creative process of understanding IT-systems and their detailed structure and functionality. Finding and documenting vulnerabilities in IT-infrastructure helps to secure them and enhance the safety of these systems.

DAST scanners are able to test software for vulnerabilities by replaying known attack patterns. In comparison to this, penetration tests are more creative approaches. A tester adopts his testing plans based on the reactions of the System Under Test and sets up his plans based on architectural details. Penetration tests in fact are able to simulate full blown realistic attacks against a System Under Test. Findings from these realistically simulated attacks reveal in detail how real world attackers could exploit existing vulnerabilities. These information are being documented in a test report to inform the responsible IT infrastructure management and propose possible mitigation strategies. [6]

If a corporate vendor decides to conduct a penetration test on its system this test can be performed either by its own employees or by a third party specializes company. Either way the goal of finding exploitable vulnerabilities needs to be specified in detail. You can refer to the example of stealing a car. By a penetration test a specialist can find out how hard it is to steal your car but this appears not to be quite specific. Stealing a car usually refers to someone driving another's car without permission and not returning it. Does a unknown driving one's car without permission refers to stealing it when it is returned next day? Does a teenager driving its parents car without parents permission refer to stealing it? The goal of finding out how hard it is to steal a car is simply not specific enough. For IT infrastructure a specific goal could for example be: Is a unauthorized party able to interfere with Confidentiality Indtegrity Availability (CIA) triad? [17]

In the next step it is important to consider the knowledge the penetration tester has about the System Under Test. Potential attackers can allocate massive resources in form of time and money if the exploitation seams worthwhile. Through providing penetration testers with information about the System Under Test they can "compensate for the disparity in resources." [17] Also penetration testers can be given partially internal access to simulate the scenario of already overcome first barriers by attackers. [17] penetration testing with different degrees of knowledge can be categorized into Black-, Grey- and White-Box penetration testing. [8]

5

AI DRIVEN PENETRATION TESTING

Tools like BurpSuite, ZAP and SQLMap already implement automated penetration testing techniques through signature-driven DAST scanning approaches. These scanners systematically map web applications, fuzz parameters with curated payloads, and detect injection vulnerabilities like XSS, SQLi, and command injection. However, these approaches are fundamentally constrained by their pattern-matching limitations that prevent the contextual reasoning, multi-step exploitation chains, and novel attack creativity required to identify complex business logic vulnerabilities and zero-day exploits. [9]

Earlier approaches to automate the process of penetration testing like ESA's PenBox leveraged Reinforcement Learning. PenBox has the capability to execute given actions like port scanning with NMap, SSH security testing with Hydra and exploitation of known vulnerabilities with Metasploit. Training landscapes were built to train PenBox to use the given actions beneficially. PenBox's training and test environment comprises 8 hosts in one network. In this rather small environment PenBox was able to compromise vulnerable hosts with 33 actions taken while the minimal requires actions would have been 4. [10]

Leveraging LLMs for penetration testing shows better results and scalability in comparison to Reinforcement Learning. Capture The Flag (CTF) Challenges from HackTheBox and VulnHub were used to evaluate the performance of GPT-4 without any technical additions. GPT-4 was prompted to advice the user to execute a specific task like running a Command Line Interface (CLI) command which's output the user prompts back to the LLM. Such a technically minimalistic setup proves that LLMs are capable of performing penetration tests as GPT-4 is able to solve 4 easy and 1 medium challenge. This benchmark was executed on 13 targets from HackTheBox and VulnHub covering OWASP Top 10 and categories easy, medium and hard. The available knowledge was entirely limited to the LLMs inherit domain knowledge. [12]

The process of penetration testing can be split up into multiple tasks which allows to deligate different tasks to different LLM agents that are specialized in certain fields. The architecture of PentestGPT introduces three agents to delegate these tasks to. One agent responsible for information condensing, one for reasoning and one for concrete task planing. This agentic approach further improves the CTF challenge completion rate. Without the agentic approach GPT-4 was able to solve 4 easy and 1 medium challenge, with the agentic setup it is able to solve 6 easy and 2 medium challenges out of the same set of challenge. The increase in CTF challenge completion rate points to the significance of the LLM orchestration. [12]

One limiting factor towards higher CTF challenge completion is the lack of up-to-date domain specific knowledge such as known vulnerabilities. The knowledge of an LLM is entangled within thousands of parameters. Retraining LLMs with up-to-date information is computationally very expensive. Retrieval-augmented generation (RAG) introduced by Lewis et al. [18] is a common approach to provide LLMs with knowledge it has not inertly build in. The additional information are stored as embeddings in a vector database. The LLM is then provided with retrieved information from the RAG's vector database. [18]

PentestAgent improves the architecture of PentestGPT by introducing domain specific knowledge on known vulnerabilities. It equips the agentic setup with a RAG. This external knowledge base makes multiple vulnerability databases accessible for PentestAgent. Through several adjustments regarding the agentic setup and the additionally introduces domain specific knowledge PentestAgent outperforms PentestGPT by far when it comes to solving CTF challenges. 11 CTF challenges were randomly selected from HackTheBox (nine easy, one medium and one hard). PentestGPT was able to solve 5 of these challenges (all easy) while PentestAgent was able to solve 9 of these challenge (7 easy, one medium and one hard). [19]

The success of PentestGPT and PentestAgent are not limited to small networks or even single hosts. AI Agents and thus LLMs are also capable of performing penetration tests in real world and real size networks. ARTEMIS is an agentic approach like PentestGPT and PentestAgent. However it was optimized and tested in a real world university network. Two versions of ARTEMIS were benchmarked against experienced human penetration testers. The first ARTEMIS version running entirely on GPT-5 came up with 11 findings out of which 55% were valid findings. The second ARTEMIS version running on multiple different LLMs was able to come up with 11 findings as well. However, 82% of them were valid. Also their severity was much higher. Both versions are able to compete with human penetration testers. The architects of ARTEMIS mention that the agents scans systems in a broad manner instead of moving lateral like experience human penetration testers do. By moving lateral human penetration testers tend to make findings with higher severities. [20]

Real world corporate applications are much more complicated and unknown than test environments like DVWA, OWASP Juice Shop, OWASP WebGoat and many CTF challenges. They have dynamic pages with JavaScript logic and proprietary, sometimes niche technology stacks. Automated testing tools need to traverse the detailed architecture of the system under test. XBOW is a LLM powered automated penetration testing tool that publicly claims to be able to work with corporate applications and all its challenges. XBOW can operate with and without access to the source code of the system under test. [21] Not many information about the detailed system structure are made public. XBOW's high position on the Hackerone leaderboard however proves that it is clearly capable of making valid findings in corporate infrastructure. [22] [23]

Corporate security experts who used XBOW on proprietary web applications shared their experiences for this term paper. To evaluate XBOW's performance, the corporate security experts used a deliberately vulnerable web application based on a proprietary technology stack as a benchmark system. The experts found that XBOW was unable to identify any vulnerabilities in this system. XBOW was provided with source code and network access to the application. According to the experts, XBOW is capable of finding vulnerabilities in other applications, but they report that the tool tends to produce a high rate of false positives. The experts also note that valid findings made by XBOW are generally low-hanging fruits from the perspective of experienced penetration testers.

Software vendors like Google implement LLM powered penetration testing tools into software security processes. Google's open-source oss-fuzz for example is a LLM powered fuzzing tool that actively found many security flaws in open source software. [24]

5.1 SECURITY CHALLENGES

LLMs are trained based on a set of data. Those datasets do not encompass the entirety of available information. As not even the entirety of information necessarily represents truth, LLMs have inherit bias from their training data. This can lead to unwanted treatment of actual vulnerabilities. A biased LLM can potentially overlook vulnerabilities. [25]

In the context of penetration testing LLMs are often times equipped with tools like web search. Prompts can be injected into the LLM's context through the access of third party webpages. In the combination with the LLM holding confidential data, the risk of leaking these information to a third party webpage rises.

6 DISCUSSION

Corporate networks span hundreds or thousands of hosts. Large corporate networks let Reinforcement Learning approaches reach their limits. The training process of Reinforcement Learning approaches like PenBox requires high fidelity training environments. [11] Automating the process of penetration testing with AI models trained through Reinforcement Learning is rather limited as capabilities of these models often concentrate on exploiting known vulnerabilities through Metasploit. The inability to adopt to dynamically changing environments makes Reinforcement Learning approaches unable to identify complex business logic vulnerabilities and zero-day exploits.

The vulnerabilities that PentestGPT, PentestAgent and ARTEMIS exploited in the benchmarks are nearly all performed based on known vulnerabilities or in controlled and unrealistic test environments. For cooperate vendors especially web application penetration testing is highly interesting. This is because web applications are highly exposed and popular. Especially PentestAgent targets at exploiting known vulnerabilities instead of investigating architecture of web applications to find new vulnerabilities. For finding new vulnerabilities in unknown web applications these tools are not made and tested for.

PentestGPT's performance was evaluated based on 13 CTF challenges and PentestAgent's performance based on 11 CTF challenges which is even less. The process of choosing these challenges is opaque. PentestGPT was evaluated on hand picket challenges and PentestAgent based on randomly selected once. Both selecting processes can influence the evaluation result.

Corporate LLM driven penetration testing tools like XBOW and oss-fuzz clearly show that LLMs are capable of finding security vulnerabilities in corporate software.

When it comes to proprietary technology stacks that may even use niche frameworks XBOW reaches it's limits digging deep enough into the inner workings of proprietary code bases. The security expert who conducted tests on XBOW conclude that XBOW lacks the capability of lateral movement required to thoroughly investigate the working principles of the applications under test. This effect is also observable in ARTEMIS's evaluation.

7 CONCLUSION AND OUTLOOK

7.1 LLMS ARE CAPABLE OF PERFORMING AUTOMATED PENETRATION TESTS

Tools like BurpSuite, ZAP and SQLMap already implement partially automated penetration testing techniques through signature-driven DAST scanning approaches. AI models trained through Reinforcement Learning are also used in these tools mainly for classification.

With the rise of LLMs into the public more and more research was conducted on leveraging LLMs for automated penetration testing. Research revealed that LLMs are already capable of performing penetration tests based on their inherit knowledge. Without any technical additions publicly available LLMs can conduct penetration tests. By providing an LLM with tool usage capabilities the process can be automated nearly entirely. Additional domain specific knowledge provided to the LLM improves the process further.

Corporate LLM driven penetration testing tools like XBOW and oss-fuzz are able to find vulnerabilities in corporate tech stacks.

7.2 HYPOTHESES EVALUATION

The hypotheses that LLMs are capable of performing automated penetration tests on corporate and proprietary applications must partially be rejected because of functional limitations.

7.3 FUNCTIONAL LIMITATIONS

XBOW's high ranking on the Hackerone leaderboard clearly proves that generally LLM powered tools are capable of finding vulnerabilities in real world bug bounty programs. [22] [23] When it comes to performing penetration tests in proprietary tech stacks or applications built with niche frameworks, automated penetration tests leveraging LLMs reach their limits. Tests with proprietary tools like XBOW revealed that even highly sophisticated tools struggle with niche and proprietary tech stacks.

7.4 FUTURE WORK

With an appropriate setup, LLMs can conduct end-to-end penetration tests. Their vulnerability identification performance remains limited on proprietary or niche technology stacks. Future research should prioritize investigating the underlying causes of these limitations and developing strategies to

enhance AI-driven testing in corporate environments. Additionally, further further research should be conducted to address and mitigate the security challenges associated with these approaches.

Part II

APPENDIX

BIBLIOGRAPHY

[1] T. Given-Wilson, E. Baranov, and A. Legay, "Building user trust of critical digital technologies," in *2020 IEEE International Conference on Industrial Technology (ICIT)*, Feb. 2020, pp. 1199–1204. DOI: 10.1109/ICIT45562.2020.9067154.

[2] L. A. Gordon, M. P. Loeb, and T. Sohail, "Market value of voluntary disclosures concerning information security1," *Management Information Systems Quarterly*, vol. 34, no. 3, pp. 567–594, Sep. 2010, ISSN: 0276-7783. DOI: 10.2307/25750692.

[3] European Union, *Regulation (eu) 2016/679 of the european parliament and of the council of 27 april 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data, and repealing directive 95/46/ec (general data protection regulation)*, Official Journal of the European Union, L 119, 4 May 2016, pp. 1–88, 2016. [Online]. Available: https://eur-lex.europa.eu/legal-content/EN/TXT/PDF/?uri=CELEX:32016R0679.

[4] A. Adeyeri and H. Abroshan, "Geopolitical ramifications of cybersecurity threats: State responses and international cooperations in the digital warfare era," *Information*, vol. 15, no. 11, 2024, ISSN: 2078-2489. DOI: 10.3390/info15110682.

[5] B. Potter and G. McGraw, "Software security testing," *IEEE Security & Privacy*, vol. 2, no. 5, pp. 81–85, Sep. 2004, ISSN: 1558-4046. DOI: 10.1109/MSP.2004.84.

[6] P. Lachkov, L. Tawalbeh, and S. Bhatt, "Vulnerability assessment for applications security through penetration simulation and testing," *Journal of Web Engineering*, vol. 21, no. 7, pp. 2187–2208, Oct. 2022, ISSN: 1544-5976. DOI: 10.13052/jwe1540-9589.2178. [Online]. Available: https://ieeexplore.ieee.org/document/10251051.

[7] Y. Stefinko, A. Piskozub, and A. Obshta, "Analysis of vulnerability characteristics for automated penetration testing," in *2024 IEEE 17th International Conference on Advanced Trends in Radioelectronics, Telecommunications and Computer Engineering (TCSET)*, Oct. 2024, pp. 449–453. DOI: 10.1109/TCSET64720.2024.10755620.

[8] N. Rane and A. Qureshi, "Comparative analysis of automated scanning and manual penetration testing for enhanced cybersecurity," in *2024 12th International Symposium on Digital Forensics and Security (ISDFS)*, Apr. 2024, pp. 1–6. DOI: 10.1109/ISDFS60797.2024.10527240.

[9] T. A. Osman, J. R. Ragasa, H. Posesione, A. David, and M. Shakiba, "Comparative analysis of penetration testing tools in assessing web application vulnerabilities," in *2025 Interdisciplinary Conference on Electrics and Computer (INTCEC)*, 2025, pp. 1–6. DOI: 10.1109/INTCEC65580.2025.11256013.

[10] A. Confido, E. V. Ntagiou, and M. Wallum, "Reinforcing penetration testing using ai," in *2022 IEEE Aerospace Conference (AERO)*, 2022, pp. 1–15. DOI: 10.1109/AERO53065.2022.9843459.

[11] J. Schwartz and H. Kurniawati, "Autonomous penetration testing using reinforcement learning," *ArXiv*, vol. abs/1905.05965, 2019. [Online]. Available: https://api.semanticscholar.org/CorpusID:155090057.

[12] G. Deng, Y. Liu, V. Mayoral-Vilches, P. Liu, Y. Li, Y. Xu, T. Zhang, Y. Liu, M. Pinzger, and S. Rass, "PentestGPT: Evaluating and harnessing large language models for automated penetration testing," in *33rd USENIX Security Symposium (USENIX Security 24)*, Philadelphia, PA: USENIX Association, Aug. 2024, pp. 847–864, ISBN: 978-1-939133-44-1. [Online]. Available: https://www.usenix.org/conference/usenixsecurity24/presentation/deng.

[13] M. Kozlovska, A. Piskozub, and V. Khoma, "Artificial intelligence in penetration testing: Leveraging ai for advanced vulnerability detection and exploitation," *ACPS*, vol. 10, no. 1, pp. 65–70, 2025. DOI: https://doi.org/10.23939/acps2025.01.065.

[14] R. A. Khan, S. U. Khan, H. U. Khan, and M. Ilyas, "Systematic mapping study on security approaches in secure software engineering," *IEEE Access*, vol. 9, pp. 19 139–19 160, 2021, ISSN: 2169-3536. DOI: 10.1109/ACCESS.2021.3052311.

[15] D. Dalaq, K. F. Daya, A. Dalaq, M. N. Arefin, and M. K. Niazi, "A systematic literature review on static application security testing (sast) tools: Evaluation, benchmarks, challenges, and future directions," in *Proceedings of the 2025 29th International Conference on Evaluation and Assessment in Software Engineering Companion*, ser. EASE Companion '25, New York, NY, USA: Association for Computing Machinery, 2025, 162–168, ISBN: 9798400718328. DOI: 10.1145/3727967.3756838. [Online]. Available: https://doi.org/10.1145/3727967.3756838.

[16] R. Singh, M. Kumar Gupta, D. R. Patil, and S. Maruti Patil, "Analysis of web application vulnerabilities using dynamic application security testing," in *2024 IEEE 9th International Conference for Convergence in Technology (I2CT)*, 2024, pp. 1–6. DOI: 10.1109/I2CT61223.2024.10543484.

[17] M. Bishop, "About penetration testing," *IEEE Security & Privacy*, vol. 5, no. 6, pp. 84–87, 2007, ISSN: 1558-4046. DOI: 10.1109/MSP.2007.159.

[18] P. Lewis, E. Perez, A. Piktus, *et al.*, "Retrieval-augmented generation for knowledge-intensive nlp tasks," in *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, Eds., vol. 33, Curran Associates, Inc., 2020, pp. 9459–9474. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2020/file/6b493230205f780e1bc26945df7481e5-Paper.pdf.

[19] X. Shen, L. Wang, Z. Li, Y. Chen, W. Zhao, D. Sun, J. Wang, and W. Ruan, "Pentestagent: Incorporating llm agents to automated penetration testing," in *Proceedings of the 20th ACM Asia Conference on Computer and Communications Security*, ser. ASIA CCS '25, New York, NY, USA: Association for Computing Machinery, 2025, pp. 375–391, ISBN: 9798400714108. DOI: 10.1145/3708821.3733882. [Online]. Available: https://doi.org/10.1145/3708821.3733882.

[20] J. W. Lin, E. K. Jones, D. J. Jasper, *et al.*, *Comparing ai agents to cybersecurity professionals in real-world penetration testing*, 2025. arXiv: 2512.09882 [cs.AI]. [Online]. Available: https://arxiv.org/abs/2512.09882.

[21] *Xbow: Autonomous offensive security platform*, https://xbow.com/, Accessed: 2026-02-20, XBOW, 2026.

[22] HackerOne, *Leaderboard: All time reputation (q1 2026, us, web app, owasp a1)*, Accessed: 2026-02-23, 2026. [Online]. Available: https://hackerone.com/leaderboard/all_time_reputation?year=2026&quarter=1&owasp=a1&country=US&assetType=WEB_APP&tab=bbp&userTypeTab=collective.

[23] Arch1t3ct, *Behind the hype: Is xbow ai really the "game-changer" they claim it to be?* Accessed: 2026-03-06, 2025. [Online]. Available: https://godaccess.substack.com/p/behind-the-hype-is-xbow-ai-really-the-game-changer.

[24] A. Happe and J. Cito, *On the surprising efficacy of llms for penetration-testing*, 2025. arXiv: 2507.00829 [cs.CR]. [Online]. Available: https://arxiv.org/abs/2507.00829.

[25] E. Hilario, S. Azam, J. Sundaram, K. Imran Mohammed, and B. Shanmugam, "Generative ai for pentesting: The good, the bad, the ugly," *International Journal of Information Security*, vol. 23, no. 3, pp. 2075–2097, 2024, ISSN: 1615-5270. DOI: 10.1007/s10207-024-00835-x. [Online]. Available: https://doi.org/10.1007/s10207-024-00835-x.